

AID 728 — Computer Vision

Course Project Report on

Traffic Rule Violation Detection

Dawood Aziz Zargar (MT2025705) Mehraan Azhar Mir (MT2025716)

Saikat Pal (MT2025723)

0 Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Constraints	3
2	System Overview	3
2.1	High-Level Architecture	4
2.2	Data Contracts Between Modules	4
2.3	Output Schema	4
3	Module 1: Two-Wheeler and Rider Detection	5
3.1	Objective	5
3.2	Model	5
3.3	Algorithm	5
3.4	Rider Association: IoP-based Two-Pass Strategy	5
3.5	Configuration Parameters	6
3.6	Output: VehicleInstance	6
4	Module 2: Helmet Detection	6
4.1	Objective	6
4.2	Model	6
4.3	Algorithm	6
4.4	Class-Name Normalisation	7
4.5	Behavioural Rules	7
4.6	Output: HelmetResult	7
5	Module 3: Licence Plate Reading	7
5.1	Objective	7
5.2	Models	7
5.3	Two-Stage Pipeline	8
5.4	Post-Processing Layers	8
5.5	Output: PlateResult	9
6	Module 4: Violation Engine	9
6.1	Objective	9
6.2	Violation Rules	9
6.3	Output: ViolationReport	9
6.4	Assignment Output Mapping	10
7	Model Budget	10
8	Implementation Details	10
8.1	Solution Entry Point	10
8.2	Robustness Guarantees	11
8.3	Submission Layout	11
9	Sample Results	12
10	Failure Cases and Limitations	13

10.1 Known Failure Modes	13
10.2 Impact on Evaluation Metrics	13
11 Improvements Implemented	13
11.1 Module 1 Improvements	13
11.2 Module 2 Improvements	14
11.3 Module 3 Improvements	14
12 Potential Future Improvements	14
13 Conclusion	15

1 Introduction

Traffic rule violations involving two-wheelers — over-capacity riding and failure to wear helmets — are major contributors to road fatalities in South Asian urban environments. Manual enforcement is limited by available personnel and reaction time; an automated system capable of processing a single street-scene photograph can supplement enforcement at scale.

This project implements an end-to-end computer vision pipeline that ingests a single RGB image and outputs, per violating motorcycle: the total rider count, the number of riders without a helmet, and the vehicle’s licence plate text. The system operates entirely offline, within a **250 MB** total model budget, and returns results in a well-defined JSON schema.

1.1 Problem Statement

Given one RGB image from a street scene, the system must:

1. Detect all two-wheelers (motorcycles / scooters).
2. Classify each detected vehicle for the following violations:
 - More than two riders on a single two-wheeler (*triple riding*).
 - One or more riders not wearing a helmet (*helmet violation*).
 - A combination of both.
3. Extract and report the vehicle registration number (licence plate) for each violating vehicle.

1.2 Constraints

Table 1: System constraints

Constraint	Limit
Total model size	≤ 250 MB
Maximum model size (single model)	< 1 B parameters
Internet access during evaluation	Disabled
Inference time	Must be reasonable per image

2 System Overview

The pipeline is decomposed into four sequential modules with cleanly defined input/output contracts. Parallelism is possible between Modules 2 and 3 since both consume only the `VehicleInstance` produced by Module 1.

2.1 High-Level Architecture

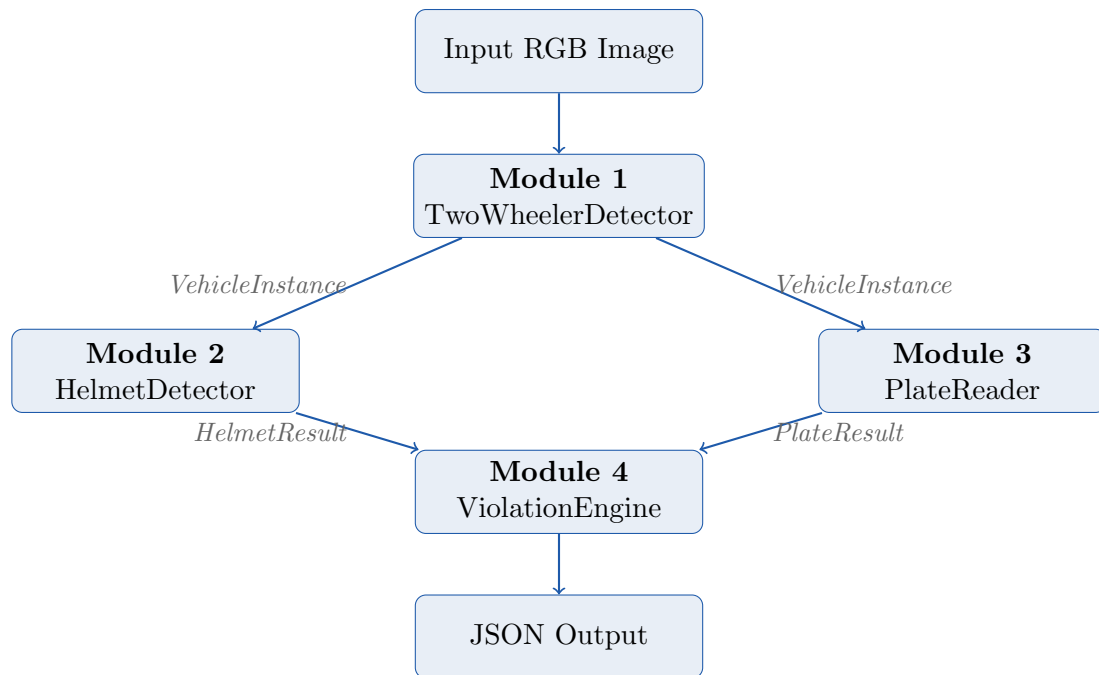


Figure 1: Four-module pipeline. Modules 2 and 3 run independently on each `VehicleInstance` produced by Module 1.

2.2 Data Contracts Between Modules

Every module communicates through typed Python dataclasses, ensuring no implicit coupling and enabling isolated unit testing of each stage.

Table 2: Inter-module data contracts

Dataclass	Produced by	Key fields
<code>VehicleInstance</code>	Module 1	crop image, rider boxes, num_riders
<code>HelmetResult</code>	Module 2	helmet_violations, per_rider_status
<code>PlateResult</code>	Module 3	license_plate, ocr_confidence
<code>ViolationReport</code>	Module 4	violations, license_plate, flags

2.3 Output Schema

```

1 {
2   "violations": [
3     {
4       "num_riders":      int,    # total riders on the vehicle
5       "helmet_violations": int,  # riders without helmets
6       "license_plate":   str     # OCR result
7     }
8   ]
9 }
```

Listing 1: Required JSON output format per image

3 Module 1: Two-Wheeler and Rider Detection

3.1 Objective

Module 1 locates every ridden two-wheeler in the image. *Parked* motorcycles (no associated rider) are explicitly dropped — they cannot incur a violation.

3.2 Model

Table 3: Model used in Module 1

Model	File	Size	Source
YOLOv8m (COCO)	yolo8m.pt	50 MB	Ultralytics official

COCO class 3 (motorcycle) and class 0 (person) are used without fine-tuning. COCO accuracy on these two classes is already strong for street scenes.

3.3 Algorithm

1. Run YOLOv8m with test-time augmentation (`augment=True`) separately for motorcycles ($\text{conf} \geq 0.25$) and persons ($\text{conf} \geq 0.40$).
2. Deduplicate overlapping motorcycle boxes with greedy NMS ($\text{IoU} \geq 0.70$).
3. Associate riders to each motorcycle in two passes.
4. For every motorcycle with at least one confirmed rider, compute a padded union bounding box (15% extra context on each side) and store the crop as `VehicleInstance.crop_image`.

3.4 Rider Association: IoP-based Two-Pass Strategy

The core difficulty is distinguishing riders from bystanders, pedestrians, or car occupants who appear at the same pixel height.

Pass 1 — Full-body IoP (primary):

$$\text{IoP}(p, m) = \frac{\text{Area}(p \cap m)}{\text{Area}(p)}$$

where p is the person bounding box and m is the motorcycle bounding box. Real riders score $\text{IoP} \in [0.40, 0.90]$; bystanders and car occupants score $\text{IoP} \in [0.05, 0.15]$. A person is assigned to a motorcycle if $\text{IoP} \geq 0.30$.

Pass 2 — Vertical-gap fallback: Handles riders whose torso extends above the motorcycle box. All three conditions must hold simultaneously:

- $\text{IoP} \geq 0.15$
- Horizontal overlap fraction ≥ 0.30
- Vertical gap = $\max(0, \text{moto_top} - \text{person_bottom}) \leq 20 \text{ px}$

The vertical-gap check replaced an older centroid-distance heuristic that failed on close-up shots of tall vehicles where the rider’s centroid drifts far from the motorcycle’s centroid.

3.5 Configuration Parameters

Table 4: Module 1 configuration parameters

Parameter	Default	Effect
MOTORCYCLE_CONF	0.25	Detection threshold for motorcycles
PERSON_CONF	0.40	Detection threshold for persons
MIN_FULL_BODY_IOP	0.30	Pass 1 IoP gate
MAX_VERTICAL_GAP_PX	20	Pass 2 vertical gap tolerance (pixels)
CROP_PADDING	0.15	Context padding fraction
TTA	True	Test-time augmentation toggle

3.6 Output: VehicleInstance

Each output object carries the motorcycle bounding box (original image coordinates), per-rider bounding boxes (both in original and crop-local coordinates), the crop image itself, and two warning flags: `small_crop_flag` (crop $< 60 \times 60$ px) and `fused_bike_warning` (one detection likely covers two real bikes).

4 Module 2: Helmet Detection

4.1 Objective

For every rider associated with a `VehicleInstance`, Module 2 determines whether a helmet is worn, producing a per-vehicle count of helmet violations.

4.2 Model

Table 5: Model used in Module 2

Model	File	Size	Source
YOLOv8m (helmet fine-tuned)	<code>yolov8m_helmet.pt</code>	50 MB	Roboflow Universe

4.3 Algorithm

1. For each rider bounding box (in crop-local coordinates), extract the **head region**: the top 45% of the rider’s bounding box with an additional 10% fractional padding, clamped to image boundaries.
2. If the head region is smaller than 20×20 px, mark as `undetectable` (counts as a violation per the safety-conservative spec).
3. Run the helmet YOLO on the head crop; apply per-class confidence thresholds (0.40 for both `helmet` and `no_helmet`).

4. Resolve the label: highest-confidence prediction wins; `no_helmet` breaks ties (fail-safe toward flagging).
5. One bad head crop never fails the entire vehicle (per-rider exception isolation).

4.4 Class-Name Normalisation

Roboflow models use inconsistent class naming across uploads. A `HELMET_CLASS_MAP` dictionary normalises all encountered variants to canonical `helmet` / `no_helmet`. Init-time validation warns if the loaded model has zero recognisable classes.

4.5 Behavioural Rules

Table 6: Helmet detection behavioural rules

Rule	Description
A	No detection on a head region $\Rightarrow$ counts as violation
B	Independent per-class confidence thresholds
C	Conflicting boxes on same head $\Rightarrow$ highest confidence wins
D	<code>small_crop_flag=True</code> input $\Rightarrow$ mark <code>low_confidence_flag</code>

4.6 Output: HelmetResult

Table 7: HelmetResult fields

Field	Meaning
<code>helmet_violations</code>	Count of riders without confirmed helmet
<code>total_riders</code>	Mirrors <code>VehicleInstance.num_riders</code>
<code>per_rider_status</code>	"helmet" "no_helmet" "undetectable" (index-aligned)
<code>low_confidence_flag</code>	True when input had <code>small_crop_flag</code>

5 Module 3: Licence Plate Reading

5.1 Objective

Module 3 locates the vehicle registration plate within each `VehicleInstance` crop and reads its text using an OCR ensemble. This is the most complex module, combining a fine-tuned plate detector with a multi-variant preprocessing and voting strategy.

5.2 Models

Table 8: Models used in Module 3: Licence Plate Reading

Model	File	Size	Source
YOLOv8s (plate detector)	<code>yolov8s_plates_final.pt</code>	21 MB	Fine-tuned (mAP50 $\approx$ 0.94)
FastPlateOCR (ONNX)	<code>global_mobile_vit_v2_ocr.onnx</code>	4.8 MB	Pretrained
OCR config	<code>global_mobile_vit_v2_ocr_config.yaml</code>	<1 KB	FastPlateOCR charset

FastPlateOCR was preferred over EasyOCR / Tesseract because it is trained specifically on licence plates, runs via ONNX at approximately 10 ms per plate on CPU, and supports fine-tuning on domain-specific data.

5.3 Two-Stage Pipeline

Stage 1 — Plate detection. YOLOv8s with TTA enabled ($\text{conf} \geq 0.15$) is run on the vehicle crop. The highest-confidence box meeting a minimum size (30×10 px) is retained.

Stage 2 — OCR ensemble. Five to seven preprocessing variants are generated from the detected plate region:

Table 9: OCR preprocessing variants

Variant	Addresses
default	Baseline padding
tight	Detector box too loose
wide	Detector box too tight
CLAHE	Low-contrast / under-exposed plates
sharp	Motion blur
deskew	Tilted plates (added when 4-corner contour found)
row_top/row_bottom	Indian 2-row format (aspect ratio < 2.5)

All variants are batched into a single ONNX inference call. The winning variant is selected by mean character probability. When multiple variants fall within 0.02 of the leader, a **per-position majority vote** picks the most common character at each position across tied variants.

5.4 Post-Processing Layers

1. **2-row concatenation:** `row_top` and `row_bottom` predictions are concatenated and entered as a `two_row` candidate.
2. **Format validation:** cleaned text must be 4–12 characters and contain at least one letter and one digit.
3. **Confusion-character rescue:** invalid outputs are retried with standard confusion-pair substitutions (Table 10).
4. **Confidence floor:** if mean char probability < 0.45 , the result is set to "UNREAD" rather than emitting a hallucinated string.

Table 10: Confusion-character substitution pairs

Direction	Pairs
Digit $\rightarrow$ Letter	0 $\rightarrow$ O, 1 $\rightarrow$ I, 2 $\rightarrow$ Z, 5 $\rightarrow$ S, 6 $\rightarrow$ G, 8 $\rightarrow$ B
Letter $\rightarrow$ Digit	O $\rightarrow$ 0, I $\rightarrow$ 1, Z $\rightarrow$ 2, S $\rightarrow$ 5, G $\rightarrow$ 6, B $\rightarrow$ 8

5.5 Output: PlateResult

Table 11: PlateResult fields

Field	Meaning
license_plate	Cleaned text, or "UNDETECTED"
plate_box	$[x_1, y_1, x_2, y_2]$ in original image coordinates
confidence	YOLO plate detection confidence
ocr_confidence	Mean character probability of the winning variant
valid_format	Passed length and alphanumeric mix checks
variant_used	Which preprocessing variant produced the result

6 Module 4: Violation Engine

6.1 Objective

Module 4 is a pure rule-based aggregation layer. It receives index-aligned lists of `VehicleInstance`, `HelmetResult`, and `PlateResult` and emits one `ViolationReport` per vehicle. No machine learning is used — keeping the violation logic separate and ML-free makes it auditable and easy to extend with future violation types.

6.2 Violation Rules

Table 12: Violation detection rules

Violation	Tag	Condition
Helmet violation	"no_helmet"	$\exists i : \text{per_rider_status}[i] \neq \text{"helmet"}$
Triple riding	"triple_rider"	$\text{num_riders} \geq 3$

Both violations can fire simultaneously on the same vehicle.

6.3 Output: ViolationReport

```

1 @dataclass
2 class ViolationReport:
3     vehicle_id          : int
4     license_plate       : str
5     plate_valid         : bool
6     plate_box           : list | None
7     motorcycle_box      : list
8     num_riders          : int
9     violations          : list[str]
10    per_rider_status     : list[str]
11    detection_confidence : float
12    ocr_confidence       : float
13    flags                : list[str]
```

Listing 2: ViolationReport dataclass

Upstream flags (`small_crop`, `fused_bike`, `low_confidence_helmet`) are echoed into `ViolationReport.f` to avoid re-deriving state that already exists upstream.

6.4 Assignment Output Mapping

Table 13: Mapping from `ViolationReport` to the required JSON schema

JSON key	Source
<code>num_riders</code>	<code>ViolationReport.num_riders</code>
<code>helmet_violations</code>	Count of entries in <code>per_rider_status</code> that are not "helmet"
<code>license_plate</code>	<code>ViolationReport.license_plate</code>

7 Model Budget

Table 14: Model budget summary

File	Mod.	Architecture	Size	Role
<code>yolov8m.pt</code>	1	YOLOv8m	50 MB	Motorcycle + person detection
<code>yolov8m_helmet.pt</code>	2	YOLOv8m	50 MB	Helmet classification
<code>yolov8s_plates_final.pt</code>	3	YOLOv8s	21 MB	Plate detection
<code>global_mobile_vit_v2_ocr.onnx</code>	3	MobileViT-v2	4.8 MB	Plate OCR
<code>global_mobile_vit_v2_ocr_config.yaml</code>	3	—	<1 KB	OCR charset
Total			≈126 MB	Within 250 MB limit

The remaining ≈ 124 MB budget could accommodate a second OCR model or a larger plate detector without violating the size constraint.

8 Implementation Details

8.1 Solution Entry Point

The `TrafficViolationDetector` class in `solution.py` serves as the thin orchestration wrapper required by the assignment evaluator:

```

1 class TrafficViolationDetector:
2     def __init__(self, model_dir="./models"):
3         self.m1 = TwoWheelerDetector(f"{model_dir}/yolov8m.pt")
4         self.m2 = HelmetDetector(f"{model_dir}/yolov8m_helmet.pt")
5         self.m3 = PlateReader(
6             model_path      = f"{model_dir}/yolov8s_plates_final.
7                 pt",
8             ocr_onnx_path   = f"{model_dir}/
9                 global_mobile_vit_v2_ocr.onnx",
10            ocr_config_path = f"{model_dir}/
11                global_mobile_vit_v2_ocr_config.yaml",
12        )

```

```

10     self.engine = ViolationEngine()
11
12     def predict(self, image_path: str) -> dict:
13         instances = self.m1.detect(image_path)
14         helmets   = detect_batch(self.m2, instances)
15         plates    = read_batch(self.m3, instances)
16         reports   = self.engine.evaluate(instances, helmets,
17                                         plates)
18         return {
19             "violations": [
20                 {
21                     "num_riders":      r.num_riders,
22                     "helmet_violations": sum(1 for s in r.
23                                             per_rider_status if s != "helmet"),
24                     "license_plate":   r.license_plate,
25                 }
26                 for r in reports
27             ]
28         }

```

Listing 3: TrafficViolationDetector class

8.2 Robustness Guarantees

- Every public method (`detect`, `read`, `evaluate`) is wrapped in a top-level `try/except`; failure on a single image returns an empty list rather than raising an exception.
- The `predict()` method is stateless: no state is carried between calls.
- All models are loaded exactly once inside `__init__`.

8.3 Submission Layout

```

1 <ROLL_NUMBER>/
2 |-- solution.py
3 |-- test_pipeline.py
4 |-- pipeline/
5 |   |-- __init__.py
6 |   |-- two_wheeler_detector.py
7 |   |-- helmet_detector.py
8 |   |-- plate_reader.py
9 |   '-- violation_engine.py
10 |-- models/
11 |   |-- yolov8m.pt
12 |   |-- yolov8m_helmet.pt
13 |   |-- yolov8s_plates_final.pt
14 |   |-- global_mobile_vit_v2_ocr.onnx
15 |   '-- global_mobile_vit_v2_ocr_config.yaml
16 |-- requirements.txt
17 |-- README.md

```

Listing 4: Submission directory structure

9 Sample Results

Figures 2 and 3 show annotated outputs produced by the pipeline on two sample street-scene images.

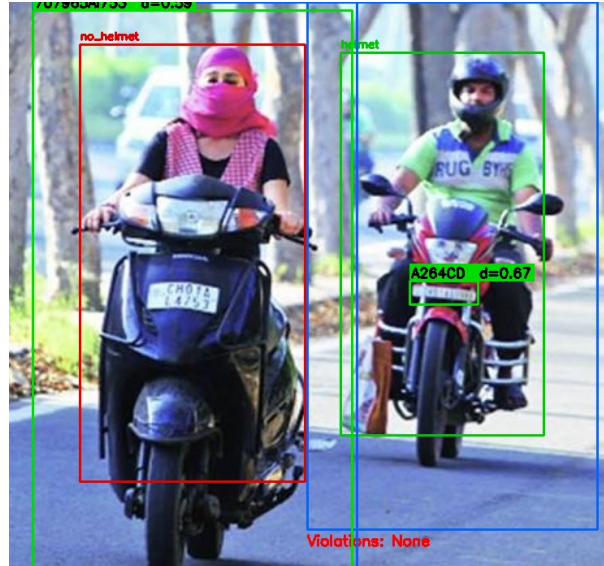


Figure 2: Single-rider detection. Left vehicle: rider flagged as `no_helmet`. Right vehicle: rider correctly classified as `helmet`

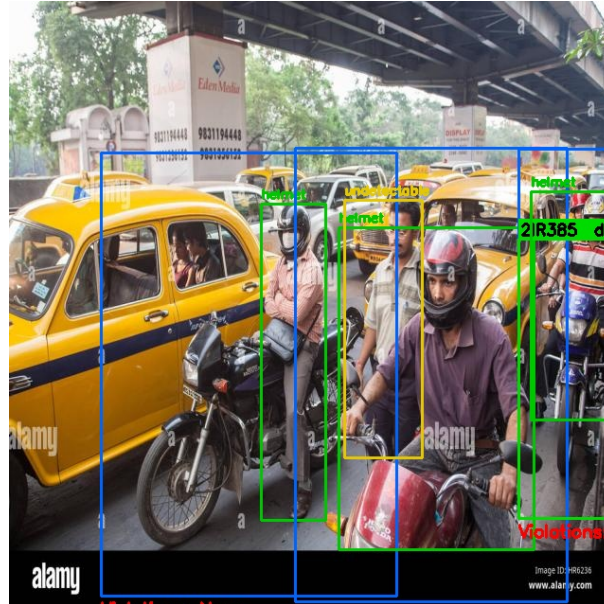


Figure 3: Dense traffic scene with three detected vehicles.

10 Failure Cases and Limitations

10.1 Known Failure Modes

Table 15: Known failure modes with root causes and mitigations

Failure mode	Root cause	Mitigation / future fix
No plate detected on distant motorcycles	Plate region too small for single-pass YOLO	Run YOLO at two scales (640 and 1280) then NMS-merge results
OCR returns "UNREAD"	Mean character confidence below threshold after all variants	Fine-tune OCR on domain data; stack a second OCR model
Fused-bike detection	Two closely spaced motorcycles form one wide detection box	Flagged via <code>fused_bike_warning</code> ; future: split on aspect ratio
Helmet misclassified on small heads	Head crop below minimum size; marked <code>undetectable</code>	Run a second higher-resolution detection pass on the vehicle crop
Non-Indian helmet styles	Training data dominated by Western full-face helmets	Fine-tune on a South Asian helmet dataset
Deskew fails without clean border	Contour-based quad detection finds no 4-corner shape	Replace with min-area-rect fit on text contours

10.2 Impact on Evaluation Metrics

- **Violation classification accuracy** is primarily degraded by the fused-bike case (all riders assigned to one vehicle) and by the `undetectable` head-region case, which conservatively counts as a violation, increasing recall at the cost of some precision.
- **OCR accuracy** is most impacted by small or distant plates and by 2-row Indian plates whose aspect ratio is not correctly identified by the split-row gate.

11 Improvements Implemented

The following improvements were made iteratively during development.

11.1 Module 1 Improvements

Table 16: Improvements made to Module 1

Fix	Description
IoP-based association	Replaced brittle containment / bottom-half overlap heuristic with full-body IoP
Vertical-gap fallback	Replaced centroid-distance heuristic; handles tall riders on close-up shots
Per-motorcycle NMS dedup	Prevents the same rider being assigned to two overlapping motorcycle boxes
TTA on both YOLO calls	Small recall boost on blurry and distant vehicles
Module exception guard	Always returns [] on any internal failure

11.2 Module 2 Improvements

Table 17: Improvements made to Module 2

Fix	Description
Class-name normalisation	Allows swapping Roboflow models without silent class-mapping failures
Init-time class validation	Warns immediately if the loaded model has zero recognisable classes
Per-class confidence thresholds	Independent gates per class, biased toward flagging violations
<code>no_helmet</code> tiebreaker	Fail-safe: equal-confidence ties resolve to a violation
Per-rider exception isolation	One bad crop never kills the entire vehicle's result

11.3 Module 3 Improvements

Table 18: Improvements made to Module 3

Layer	Problem solved
YOLO TTA	Recovers small or blurry plates missed by a single pass
5 preprocessing variants	Covers loose/tight detector boxes, low contrast, and motion blur
Batched ONNX inference	All variants processed in one call — near-zero additional cost
Per-position majority vote	Character-level consensus across tied variants
Deskew via 4-corner contour	Largest single accuracy gain on tilted plates
2-row split-row OCR	Enables reading Indian state-code + registration-number plates
Confusion-character rescue	Recovers digit-only or letter-only invalid outputs
Confidence floor (0.45)	Emits "UNREAD" rather than hallucinated text

12 Potential Future Improvements

Listed in priority order (highest impact first):

1. **Fine-tune FastPlateOCR** on synthetic Indian plates with heavy blur and JPEG-artifact augmentations.
2. **Multi-scale plate detection** — run YOLO at two image sizes and NMS-merge results to recover small and distant plates.
3. **Second OCR model ensemble** — load a second FastPlateOCR architecture and average predictions; near-free on CPU.
4. **Re-detect persons inside crop** — a second person-detection pass at higher resolution on each vehicle crop to catch pillion riders missed in the full-image pass.

5. **Better deskew** — fit a min-area-rect to text contours instead of finding the largest 4-corner contour.
6. **Indian-plate format validator** — constrain OCR output to the regex $\wedge [A-Z] \{2\} [0-9] \{1,2\} [A-Z] \{1,2\} [0-9] \{4\} \$$.
7. **Fine-tune helmet model on South Asian data** — improve recall on half-shell and turban-style helmets common in the region.

13 Conclusion

This report presented a modular, four-stage computer vision pipeline for automated traffic violation detection on two-wheelers. The system:

- Detects motorcycles and their riders using IoP-based association robust to both top-down and street-level camera angles.
- Classifies helmet compliance per rider with a safety-conservative *no-detection* $\Rightarrow$ *violation* policy.
- Reads vehicle registration plates with a multi-variant OCR ensemble, deskew correction, and 2-row Indian plate support.
- Aggregates violations through a pure rule-based engine with full upstream flag propagation.

Sample outputs on real street-scene images demonstrate that the system correctly identifies helmet violations and reads licence plates in clear conditions, while performance degrades gracefully in dense traffic and heavy occlusion scenarios.

The total model footprint is ≈ 126 MB, well within the 250 MB constraint. All public methods are wrapped in exception guards. The pipeline is designed for modularity: each component can be improved, replaced, or fine-tuned independently without affecting the others.